Pandas 라이브러리 함수 정리 및 map, apply, transform 핵심 가이드

Pandas의 핵심 함수 목록과 데이터 처리에 필수적인 map, apply, transform의 명확한 비교 및 실무 예제를 정리한 레퍼런스 가이드입니다.

1. 이미지 목차 기반 Pandas 함수 종합 정리

1.1 데이터 추출 및 인덱스 제어

함수명	적용 대상	기본 축 (Axis)	설명 및 특징
loc	Series, DataFrame	N/A	라벨 기반 인덱서. 행/열 라벨을 기준으로 슬라이싱(끝점 포함) 및 조건 필터링
iloc	Series, DataFrame	N/A	정수 위치 기반 인덱서. 0부터 시작하는 위치 인덱스 기준 슬라이싱(끝점 미포함)
drop	Series, DataFrame	axis=0 (행)	레이블 기반 행 또는 열 삭제 (axis=0: 행 삭제, axis=1: 열 삭제)
filter	Series, DataFrame	axis=1 (열)	이름(레이블) 기준 필터링 (items, like, regex 지원)
select_dtypes	DataFrame	N/A	데이터 타입을 기준으로 특정 열 선택 (include, exclude 매개변수 사용)
set_index / reset_index	DataFrame	N/A	특정 열을 인덱스로 설정하거나, 기존 인덱스를 일반 열로 리셋
reindex	Series, DataFrame	axis=0 (행)	새 인덱스 기준으로 데이터 재배열 (axis=0: 행, axis=1: 열)
rename / rename_axis	Series, DataFrame	axis=0 / axis=1	컬럼/인덱스 라벨 이름 변경 또는 축 자체의 이름 변경

1.2 데이터 정제 및 결측치/중복 처리

함수명	적용 대상	기본 축 (Axis)	설명 및 특징
sort_values	Series, DataFrame	axis=0 (행)	지정한 컬럼의 값 기준으로 정렬 (ascending으로 오름/내림차순 제어)
isna / isnull	Series, DataFrame	N/A	결측치 여부를 확인하여 Boolean 마스크 반환
dropna	Series, DataFrame	axis=0 (행)	결측치를 포함한 행 또는 열 삭제 (axis=0: 행, axis=1: 열)
fillna	Series, DataFrame	axis=0 (행)	결측치를 특정 값 또는 ffill/bfill 방식으로 대체
interpolate	Series, DataFrame	axis=0 (행)	선형 또는 다양한 수치 보간법을 사용하여 결측치 대체
clip	Series, DataFrame	axis=0 (행)	지정한 상/하한 임계값을 벗어나는 수치 데이터를 제한(클리핑)
duplicated / drop_duplicates	Series, DataFrame	axis=0	중복 데이터 식별 및 중복 행 제거
astype / to_numeric	Series, DataFrame	N/A	데이터 타입 명시적 변환 및 수치형 타입 안전 변환

1.3 데이터 결합, 변환 및 가공

함수명	적용 대상	기본 축 (Axis)	설명 및 특징
concat	Series, DataFrame	axis=0 (세로)	여러 데이터프레임/시리즈를 물리적으로 결합 (axis=1: 가로 결합)
merge / merge_asof	DataFrame	N/A	공통 키 기준 병합 또는 시계열/수치 정렬 기준 가장 가까운 값 병합

함수명	적용 대상	기본 축 (Axis)	설명 및 특징
combine_first	Series, DataFrame	N/A	첫 번째 객체의 결측치를 두 번째 객체의 동일 위치 값으로 채움
pipe	Series, DataFrame	N/A	연속적인 사용자 정의 함수를 메서드 체이닝 방식으로 연결
explode	DataFrame	N/A	리스트 형태의 원소를 풀어 여러 행으로 전개
pivot_table	DataFrame	N/A	행, 열, 집계 함수를 지정하여 요약 피벗 테이블 생성
stack / unstack	Series, DataFrame	N/A	컬럼 구조와 인덱스 레벨 간 계층 구조 재배치(피벗팅)
cut / qcut	Series	N/A	동일 간격(cut) 또는 분위수 기준(qcut)으로 수치형 데이터 범주화

1.4 시계열 및 문자열 특수 함수

구분	함수/접근자	설명 및 특징
시계열	**resample**	DatetimeIndex 기반 시계열 주기를 변경하여 리샘플링 및 그룹 집계
시계열	**rolling / expanding**	이동평균, 누적 합계 등 지정된 윈도우 크기 기반 통계 계산
문자열	**str 접근자**	str.contains, str.extract, str.split 등 Series 내 원소별 문자열 처리
날짜/시간	**dt 접근자**	dt.strftime, dt.year, dt.day_name 등 Datetime Series 속성 추출 및 포맷팅

2. map, apply, transform 핵심 비교 및 구분 기준

2.1 개념 및 특성 비교

구분	map	apply	transform
적용 대상	Series 전용	Series, DataFrame	Series, DataFrame, GroupBy
주요 기능	원소별 1:1 매핑 및 단순 치환	다중 열 참조, 복잡한 로직 처리	원본 크기(Shape) 유지 연산 (Broadcasting)
전달 인자	dict, Series, single-arg function	function, extra args	function, string (e.g., 'mean')
반환 형태	Series	Series, DataFrame, Scalar	원본과 동일한 차원의 Series/DataFrame
성능/특징	가장 가볍고 속도가 빠름	유연성이 높으나 상대적으로 속도가 느림	GroupBy와 연계하여 그룹 통계량 반영 시 필수

3. map, apply, transform 활용 예제 (각 10선)

3.1 map 함수 활용 예제 10선

- **이진 분류 인코딩**: df['sex'].map({'male': 0, 'female': 1})
- **레이블 명칭 변경**: df['pclass'].map({1: 'First', 2: 'Second', 3: 'Third'})
- **결측치/기본값 처리**: df['pclass'].map(lambda x: pclass_map.get(x, 'Normal'))
- **수치 포맷팅**: df['fare'].map(lambda x: f"{x:.2f}$")
- **단위 접미사 추가**: df['age'].map(lambda x: f"{int(x)}세")
- **특정 약어 치환**: df['embarked'].map({'S': 'Southampton', 'C': 'Cherbourg', 'Q': 'Queenstown'})
- **불리언 텍스트 변환**: df['survived'].map({0: '사망', 1: '생존'})
- **대문자/소문자 변환**: df['sex'].map(lambda x: x.upper())
- **카테고리 단순화**: df['pclass'].map({1: 'High', 2: 'Low', 3: 'Low'})
- **단위 환율 곱셈 스케일링**: df['fare'].map(lambda x: x * 1300)

3.2 apply 함수 활용 예제 10선

- **다중 조건 파생 변수 (axis=1)**: 나이와 성별 조건 결합 분류
- **행 단위 수치 연산 (axis=1)**: df.apply(lambda x: x['fare'] / x['pclass'], axis=1)
- **수치 로그 변환**: df['fare'].apply(lambda x: np.log1p(x) if x > 0 else 0)
- **문자열 조건부 매핑 함수 적용**: 복합 텍스트 파싱 및 분류 함수 적용
- **외부 파라미터 전달**: df['fare'].apply(check_expensive, threshold=100)
- **데이터 유효성 검사**: 각 원소의 정수/유효값 여부 확인
- **문자열 길이 계산**: df['sex'].apply(len)
- **다중 조건 이상치 탐지 (axis=1)**: 여러 열의 신호 지표를 조합한 이상치 검출
- **복잡한 수치 구간 사용자 정의 함수 적용**: 사용자 정의 비선형 구간 분할
- **다중 열 가중치 종합 점수 산출 (axis=1)**: 각 컬럼별 가중치를 적용한 종합 스코어 계산

3.3 transform 함수 활용 예제 10선

- **그룹 평균과의 차이**: df.groupby('pclass')['fare'].transform(lambda x: x - x.mean())
- **그룹 평균으로 결측치 채우기**: df['age'].fillna(df.groupby(['sex', 'pclass'])['age'].transform('mean'))
- **그룹 내 비중 계산**: df.groupby('pclass')['fare'].transform(lambda x: x / x.sum())
- **그룹 내 Z-Score 표준화**: df.groupby('sex')['fare'].transform(lambda x: (x - x.mean()) / x.std())
- **그룹 내 Min-Max 스케일링**: df.groupby('pclass')['age'].transform(lambda x: (x - x.min()) / (x.max() - x.min()))
- **그룹 데이터 개수(Count) 표시**: df.groupby('embarked')['embarked'].transform('count')
- **그룹 내 누적 합계 (Cumsum)**: df.groupby('pclass')['fare'].transform('cumsum')
- **그룹 중앙값 대비 크기 비교 플래그**: df.groupby('sex')['fare'].transform(lambda x: x > x.median())
- **그룹 내 최댓값과의 편차**: df.groupby('pclass')['age'].transform(lambda x: x.max() - x)
- **그룹 내 백분위 순위 (Percentile Rank)**: df.groupby('embarked')['fare'].transform(lambda x: x.rank(pct=True))

4. groupby 및 lambda 연계 활용 정리

4.1 groupby + transform + lambda

- x는 그룹별로 분할된 **Series 전체**를 의미합니다.
- 원본 데이터의 행 길이를 그대로 유지하면서 그룹 통계량 기반의 정규화, 편차 계산, 특정 그룹 평균 기반 결측치 채우기에 사용됩니다.

4.2 groupby + apply + lambda

- x는 각 그룹 단위의 **DataFrame 또는 Series 덩어리**를 나타냅니다.
- 그룹별 상위 N개 추출(nlargest), 복합 지표 계산, 그룹별 가중 평균 계산 등 차원이 변경되거나 복잡한 연산에 매우 유용합니다.

4.3 groupby 집계 결과 + map + lambda

- groupby 객체 자체에는 map 메서드가 직접 존재하지 않습니다.
- 따라서 사전 계산된 그룹 통계량을 to_dict()로 변환한 뒤, 원본 컬럼에 map()을 통해 주입합니다. 외부 참조 통계량 매핑 시 빠른 연산 속도를 보장합니다.

5. pandas 버전 호환 팁: Categorical groupby FutureWarning 해결

5.1 문제 현상

tips.groupby('day')['total_bill'].mean() 실행 시 아래와 같은 경고 메시지가 발생할 수 있습니다.

- FutureWarning: The default of observed=False is deprecated...

5.2 발생 원인

범주형(Categorical) 변수로 그룹화할 때, 데이터에 실제 존재하지 않는 범주까지 그룹에 포함할지 여부(observed=False)의 Pandas 기본값이 향후 버전에서 True로 전환될 예정이기 때문입니다.

5.3 해결책

그룹화 시 observed 옵션을 명시적으로 전달하여 경고를 방지하고 동일한 동작을 유지할 수 있습니다.# observed 옵션을 명시적으로 지정하여 경고 차단

```
tips.groupby('day', observed=False)['total_bill'].mean().to_dict()
```